

Lenguaje Python

Cursada 2026

Clase 3: expresiones lambda - manejo de archivos

Repaso 1

Problema:

Tenemos que procesar los datos de los estudiantes de las tres comisiones de Python. Por cada estudiante contamos con su DNI, nombre, comisión, puntos de teoría, puntos de práctica y ciudad de procedencia. ¿Qué estructura de las vistas sería la más adecuada para almacenar estos datos si tenemos las siguientes situaciones?

1. Sólo necesitamos procesar la información de los estudiantes por cada comisión por separado.
2. Sólo necesitamos tener un listado de todos los estudiantes ordenado alfabéticamente.
3. Necesitamos generar un listado de las ciudades de procedencia de los estudiantes.

Posibles soluciones

Si sólo necesitamos procesar la información de los estudiantes por cada comisión por separado.

1. Diccionario con clave = la comisión

```
In [ ]: students = {  
    'info_TM': [  
        {'DNI': 12345, 'name': 'Guido Van Rossum', 'pts_teoría': 100, 'pts_práct': 80},  
        {'DNI': 11111, 'name': 'Ada Lovelace', 'pts_teoría': 100, 'pts_práct': 90},  
    ],  
    'info_TT': [  
        {'DNI': 12345, 'name': 'Alan Turing', 'pts_teoría': 100, 'pts_práct': 80},  
        {'DNI': 11111, 'name': 'Joan Clarke', 'pts_teoría': 100, 'pts_práct': 90},  
    ],  
    'CDO': [  
        {'DNI': 12345, 'name': 'Richard Stallman', 'pts_teoría': 100, 'pts_práct': 80},  
        {'DNI': 11111, 'name': 'Katherine Johnson', 'pts_teoría': 100, 'pts_práct': 90},  
    ],  
}
```

```
In [ ]: students['info_TM']
```

Posibles soluciones

Sólo necesitamos tener un listado de todos los estudiantes ordenado alfabéticamente.

2. Lista/tupla con todos los datos - ¿En qué casos elegimos una lista o una tupla?

```
In [ ]: students = [
    {'DNI': 12345, 'name': 'Guido Van Rossum', 'pts_teoría': 100, 'pts_pr
    {'DNI': 11111, 'name': 'Ada Lovelace', 'pts_teoría': 100, 'pts_pr
    {'DNI': 12345, 'name': 'Alan Turing', 'pts_teoría': 100, 'pts_pra
    {'DNI': 11111, 'name': 'Joan Clarke', 'pts_teoría': 100, 'pts_pra
    {'DNI': 12345, 'name': 'Richard Stallman', 'pts_teoría': 100, 'pt
    {'DNI': 11111, 'name': 'Katherine Johnson', 'pts_teoría': 100, 'p
]
```

```
In [ ]: names = [student['name'] for student in students]
names
```

Ahora ordenamos los nombres de personas

```
In [ ]: names.sort()
names
```

¿Cambió la estructura **names**? ¿Y si no queremos que cambie?

```
In [ ]: names
```

```
In [ ]: #names = [student['name'] for student in students]
sorted(names)
```

- **sort()**: es un método de lista que ordena los elementos in-place (modifica la lista original) y no devuelve nada.
- **sorted()**: es una función predefinida que devuelve una lista ordenada a partir de cualquier iterable, sin alterar el original.

Posibles soluciones

Necesitamos generar un listado de las ciudades de procedencia de los estudiantes.

3. Conjunto

```
In [ ]: students = [
    {'DNI': 12345, 'name': 'Guido Van Rossum', 'pts_teoría': 100, 'pt
    {'DNI': 11111, 'name': 'Ada Lovelace', 'pts_teoría': 100, 'pts_pr
    {'DNI': 12345, 'name': 'Alan Turing', 'pts_teoría': 100, 'pts_pra
    {'DNI': 11111, 'name': 'Joan Clarke', 'pts_teoría': 100, 'pts_pra
    {'DNI': 12345, 'name': 'Richard Stallman', 'pts_teoría': 100, 'pt
    {'DNI': 11111, 'name': 'Katherine Johnson', 'pts_teoría': 100, 'p
]
```

```
cities = set([student['city'] for student in students])
cities
```

Repaso 2

¿Qué imprime la función **show_string** en la primera invocación? ¿Y en la segunda?

```
In [ ]: processing_type = "invertido"
```

```
In [ ]: def show_string(sentence, order=processing_type):
        """ This function returns the string according to the order parameter

        return sentence[::-1] if order == "invertido" else sentence[:]
```

```
In [ ]: show_string("Hola")
```

```
In [ ]: processing_type = "normal"
```

¿Qué pasa ahora si vuelvo a invocar a la función?

```
In [ ]: show_string("Hola")
```

IMPORTANTE:

Los valores por defecto de los parámetros se evalúan **una única vez** cuando se define la función.

Repaso 3

¿Cuál de las siguientes invocaciones son correctas?

```
In [ ]: def send_message(message, to_person, from_person="alguien", subject="Cons
        """ This function displays a message...
        Arguments:
        to_person: string that represents the recipient of the message
        from_person: string that represents the person who sends the message
        subject: that represents the subject of the message
        """

        print(f"""
            Origen: {from_person}
            Destino: {to_person}
            Asunto: {subject}
            Mensaje: {message}
            """)
```

```
In [ ]: #send_message()
        #send_message("Profe", "¿Ya terminamos?", "la clase", "súplica")
        #send_message("¿Cuánto falta para que termine la clase?", "Profe", subject
        #send_message(from_person="La clase", to_person="Claudia", message="¿Ya t
```

Repaso 4

¿Qué diferencias hay entre las siguientes funciones?

```
def my_args(*args):
    ....
def my_args1(**kwargs):
    ....
def my_args2(a, b, /, *, exacto=False):
```

- **args** es una **tupla** que representa a los parámetros pasados.
- **kwargs** es un **diccionario** que representa a los parámetros pasados.
- Se puede organizar los parámetros
 - **Parámetros posicionales exclusivos** (antes de `/`): los argumentos para estos parámetros deben pasarse únicamente por posición. No se puede usar el nombre del parámetro en la llamada.
 - **Parámetros con nombre exclusivos** (después de `*`): los argumentos para estos parámetros deben pasarse obligatoriamente usando su nombre (keyword arguments). No se pueden pasar de forma posicional.

¿Cuál de las siguientes invocaciones son correctas?

```
my_args()
my_args(1)
my_args((1,2))
my_args([1,2], "Metallica", 3.2)

my_args1()
my_args1(1, 2)
my_args1(band1= 'Nirvana', band2="Foo Fighters", band3="AC/DC")

my_args2(10, 3, exacto=True)
my_args2(a=10, b=2)
my_args2(a=10, b=2, exacto=True)
```

DESAFÍO 1

Queremos implementar una función que dada una cadena de texto, retorne las palabras que contiene en **orden alfabético**.

```
In [ ]: # Una posible solución
def sort1(sentence):
    """ Example using method sort"""

    words = sentence.split()
    words.sort(key=str.lower)
```

```
#words.sort()
return words
```

```
In [ ]: print(sort1("Hoy puede ser un gran día. "))
```

¿De qué tipo es el valor enviado como parámetro a **key**?

Otra forma

```
In [ ]: def sort2(sentence):
        """ Example using function sorted"""

        words = sentence.split()
        return sorted(words, key=str.lower)
```

```
In [ ]: print(sort2("Hoy puede ser un gran día. "))
```

DESAFÍO 2

Queremos implementar una función que, dada una lista de canciones, nos retorne la **lista ordenada de acuerdo al intérprete**. Tenemos registrada la siguiente info: **tema, intérprete, año, categoría**.

```
In [ ]: my_music = [
        ("Bohemian Rhapsody", "Queen", "1975", "rock"),
        ("Thunder Road", "Bruce Springsteen", "1975", "rock"),
        ("Imagine", "John Lennon", "1971", "rock"),
        ("Locura y Realidad", "Las Pastillas del Abuelo", "2007", "rock"),
        ("Ascenso", "Jauría", "2010", "rock"),
        ]
```

```
In [ ]: # Posible solución
```

Analicemos esta solución

```
In [ ]: def sort3(music):
        """ This function .... """

        return sorted(music, key=lambda elem: elem[1])
```

```
In [ ]: for elem in sort3(my_music):
        print(elem)
```

RECORDEMOS: **key** espera una función.

¿Qué son las expresiones lambda?

- Son funciones anónimas.

lambda args : expression

- +Info

```
In [ ]: lambda a, b: a+b
lambda a, b=1: a+b
```

```
In [ ]: lambda a, b=1: a+b

def my_sum(a, b=1):
    return a+b
```

Algunos ejemplos de uso

```
In [ ]: actions_list = [lambda x: x*2, lambda x: x*3]
```

¿Qué tipo de elementos contiene la lista?

```
In [ ]: param = 4

for action in actions_list:
    print(action(param))
```

Un ejemplo de la documentación oficial

```
In [ ]: def make_incrementor(n):
        return lambda x: x+n

f = make_incrementor(2)
g = make_incrementor(6)
```

```
In [ ]: print(f(42), g(42))
print(make_incrementor(22)(33))
```

¿Cuál es el tipo de **f** y **g**? ¿Por qué es correcto el último **print**?

Veamos algunos usos comunes de estas expresiones

La función map

```
In [ ]: def double(x):
        return 2*x
```

```
In [ ]: numbers = [1, 2, 3, 4, 5, 6, 7]
doubles = map(double, numbers)
```

```
for elem in doubles:
    print(elem, end=" ")
```

La función filter

```
In [ ]: def is_even(x):
        return x%2 == 0
```

```
In [ ]: numbers = [1, 2, 3, 4, 5, 6, 7]
        even_numbers = filter(is_even, numbers)

        for elem in even_numbers:
            print(elem, end=" ")
```

map y filter con lambda

```
In [ ]: numbers = [1, 2, 3, 4, 5, 6, 7]

        double_numbers = map(lambda x: 2*x, numbers)
        even_numbers = filter(lambda x: x%2 == 0, numbers)
```

```
In [ ]: for elem in double_numbers:
        print(elem, end=" ")
```

La función reduce

Se aplica una función con los dos primeros elementos de la secuencia iterable, generando un resultado parcial. Luego, aplica nuevamente la función con este resultado parcial y el tercer elemento, repitiendo lo mismo para todos los elementos.

```
In [ ]: from functools import reduce

        numbers = [1, 2, 3, 4, 5, 6, 7]

        reduce(lambda a, b: a + b, numbers)
        min_value = reduce(lambda a, b: a if a < b else b, numbers)
```

- ¿Cuántos elementos se obtienen luego de aplicar reduce?
- ¿Qué sucede si la secuencia está vacía?

```
In [ ]: reduce(lambda a, b: a + b, numbers)
```

¿Qué dice la PEP 8 respecto al uso de lambda?

Si queremos definir la función **double**, por ejemplo, usemos **def** y no **lambda**.

```
# Si
def double(x):
    return 2*x

# No
double = lambda x: 2*x
```

Y en estos casos...

```
In [ ]: numbers = [1, 2, 3, 4]

doubles = [2*x for x in numbers]
#doubles = map(lambda x: 2*x, numbers)

for elem in doubles:
    print(elem, end=" ")
```

```
In [ ]: numbers = [1, 2, 3, 4]

pares = [x for x in numbers if x%2 == 0]
#pares = filter(lambda x: x%2 == 0, numbers)
for elem in pares:
    print(elem, end=" ")
```

```
In [ ]: from functools import reduce

numbers = [1, 2, 3, 4, 5, 6, 7]
sum(numbers)
min(numbers)

#reduce(lambda a, b: a + b, numbers)
#reduce(lambda a, b: a if a < b else b, numbers)
```

Entonces...¿usamos lambda?

De los ejemplos vistos en la clase...

```
def make_incrementor(n):
    return lambda x: x+n

def sort3(music):
    """ Usamos sorted con una expresión lambda"""

    return sorted(music, key=lambda elem: elem[0])
```


DESAFIO 3

Tenemos una lista de temperaturas en grados Farenheit y queremos generar otra lista con las temperaturas en grados Celcius.

Recordemos que la fórmula es: $C = (F - 32) / 1.8$

```
In [ ]: # Posible solución SIN lambda
fahrenheit = [32, 50, 68, 212]
celsius = [(f - 32)/1.8 for f in fahrenheit]
celsius
```

```
In [ ]: # Posible solución CON lambda
fahrenheit = [32, 50, 68, 212]
celsius = list(map(lambda f: (f - 32)/1.8, fahrenheit))
celsius
```

DESAFIO 4

Escribir un código USANDO expresiones lambda que, con los siguientes datos correspondientes a docentes, genere un listado con el siguiente formato.

nombre: claudia apellido: banchoff, titulo: Prof
 nombre: Sofía apellido: Martin, titulo: Prof

Listado:

Prof. BANCHOFF, Claudia
 Prof. MARTIN, Sofía

```
In [ ]: # Posible solución
personas = [{'nombre': 'Claudia', 'apellido': 'Banchoff', 'titulo': 'Prof'},
             {'nombre': 'Sofía', 'apellido': 'Martin', 'titulo': 'Prof'}]
listado = map(lambda p : f"{p['titulo']}. {p['apellido'].upper()}. {p['no",
for elem in listado:
    print(elem)
```

CONSIGNA: tarea para el hogar

Queremos procesar una lista de posteos en X, contabilizando las apariciones de los hashtag incluidos en los posteos. Ejemplo (extraídos el 26 de marzo):

```
posts = [
    ""
```

```

    post_The only Claude Code project structure you'll
ever need 📁
    Credit: codewithbrij
    #ClaudeCode #Claude #Code #Python #PythonProgramming
#Coding #Programming
    """
    """
    <Daily Python Challenge>
    Comment your answer now ⬇️
    Ready to level up your coding game? 🚀
    #Python #PythonProgramming #Coding #Programming
#PythonChallenge #PythonQuiz
    """
    """

    Take a few minutes to complete the 2026 Python
Developers Survey and help us map out an accurate
landscape of the Python community! #python
#pythondevsurvey
    """
    """

    30 Important AI Concepts
    #Python #PythonProgramming #Coding #Programming
#DataScience #MachineLearning #AI #DeepLearning
    """
    """
]

```

Debería obtener: {'#python': 4, '#pythonprogramming': 3, '#coding': 3, '#programming': 3, '#claudecode': 1, '#ai': 1, '#claude': 1, '#pythonquiz': 1, '#deeplearning': 1, '#datascience': 1, '#code': 1, '#pythondevsurvey': 1, '#pythonchallenge': 1, '#machinelearning': 1}

Solución de la IA

La solución dada por la IA es la siguiente. ¿Les parece correcta? Piensen en una solución alternativa.

- Los que quieran, pueden subir el código a su repositorio en GitHub y compartir el enlace a las cuentas [@clauBanchoff](#) y [@sofiamartin](#).

In []: *# SOLUCIÓN DE Gemini*

```

posts = [
    """post_The only Claude Code project structure you'll ever need 📁
    Credit: codewithbrij
    #ClaudeCode #Claude #Code #Python #PythonProgramming #Coding #Program
    """<Daily Python Challenge>
    Comment your answer now ⬇️
    Ready to level up your coding game? 🚀
    #Python #PythonProgramming #Coding #Programming #PythonChallenge #Py
    """Take a few minutes to complete the 2026 Python Developers Survey!
    """30 Important AI Concepts
    #Python #PythonProgramming #Coding #Programming #DataScience #Machine
]

```

```
# 1. Extraer todos los hashtags en una sola lista plana
# Usamos un list comprehension con un filtro lambda conceptual
todas_las_palabras = " ".join(posts).split()
hashtags = list(filter(lambda w: w.startswith("#"), todas_las_palabras))

# 2. Normalizar a minúsculas
hashtags_clean = list(map(lambda h: h.lower(), hashtags))

# 3. Contar frecuencias usando un diccionario
# Tip: set(hashtags_clean) nos da los hashtags únicos para iterar
conteo = {h: hashtags_clean.count(h) for h in set(hashtags_clean)}

# 4. Ordenar por los más usados (opcional pero muy útil)
ranking = dict(sorted(conteo.items(), key=lambda item: item[1], reverse=True))

print(ranking)
```

Seguimos ...